

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Analytical Problem Solving

Course Code:	CSA03	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	Manoj M	Reg. No.:	192511060
Section / Batch:	CSE	Date:	30.07.2026

Code of Conduct

I Manoj M(192511060)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure	List Operations	1	20
Search Data Structure	Binary Search	2	20
Stack Data Structure	Stack Operations	3	20
Arithmetic Expression Evaluations	Stack Notations	4	20
Queue Data Structures	Priority Queue	5	20
GRAND TOTAL:		100 Marks	

Annexure A – Analytical Problem Solving

1. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list. (a) L1.append (10); L1.append (20); L1.append (15); (b) L2.append (10); L2.append (20); L2.append (15); L2.moveToStart (); L2.insert (39); L2.next (); L2.insert (12). (20 Marks)
2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail. (20 Marks)
3. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(),Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. (20 Marks)
4. Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to the following expressions
 - a. Prefix expression
 - b. Postfix expression. (20 Marks)
5. A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

①

a) i) L1.append(10);

Head $\rightarrow$ 10 $\rightarrow$ NULL
 $\uparrow$
curr

ii) L1.append(20);

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ NULL
 $\uparrow$
curr

iii) L1.append(15);

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 15 $\rightarrow$ NULL
 $\uparrow$
curr

b) i) L2.append(10);

Head $\rightarrow$ 10 $\rightarrow$ NULL
 $\uparrow$
curr

ii) L2.append(20);

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ NULL
 $\uparrow$
curr

iii) L2.append(15);

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 15 $\rightarrow$ NULL
 $\uparrow$
curr

iv) L2.moveToStart();

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 15 $\rightarrow$ NULL
 $\uparrow$
current

vi) L2.Next();

Head → 39 → 10 → 20 → 15 → NULL
 ↓
 cut

vii) L2.Insert(12);

Head → 39 → 12 → 10 → 20 → 15 → NULL

Final Values:

a) L1:

Head → 10 → 20 → 15 → NULL

b) L2:

Head → 39 → 12 → 10 → 20 → 15 → NULL

- ② → Binary Search is a searching technique used on sorted arrays. It repeatedly divides the search interval into two halves until the element is found or the interval becomes empty.

→ Time & Space Complexity:

Best case: $O(1)$

Average & worst case: $O(\log n)$

→ Advantages:

- i) Very fast for large sorted data.
- ii) Requires only $\log_2(n)$ comparisons.
- iii) Efficient memory usage in iterative version.

→ Disadvantages:

- i) Works only on sorted data.
- ii) Not suitable for linked list because random access is needed.

→ General Algorithm:

Binary Search (A , key)

low = 0

high = $n - 1$

while (low <= high)

mid = $(low + high) / 2$

if ($A[mid] == key$)

```
    return mid
else if (key < A[mid])
    high = mid - 1
else
    low = mid + 1
```

```
return -1
```

Implementation in C:

```
#include <stdio.h>

int binarySearch (int arr[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key) {
            return mid;
        }
        else if (key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}
```

⇒ Conclusion:

→ Binary search is one of the most efficient searching algorithms for sorted arrays.

⇒ In the given array 3 cases:

- i) 111 is found at index 1 (2nd pos)
- ii) 888 is found at index 9
- iii) 999 is not present in array

③ Step-by step operation execution:

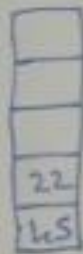
Operation	Stack (Bottom → Top)	Top
Initial	Empty	-1
Push(45)	45	0
Push(22)	45, 22	1
Push(11)	45, 22, 11	2
Push(77)	45, 22, 11, 77	3
Pop()	45, 22, 11	2
Push(44)	45, 22, 11, 44	3
Push(33)	45, 22, 11, 44, 33	4

POP() 45, 22, 11, 44 3

POP() 45, 22, 11 2

POP() 45, 22 1

Final stack:



Top = 1 Top element = 22

max-size = 5

⑤ Given:

Infix: $((a+b)+c*(d+e)+f)*(g+h)$

a) Infix to prefix

Reverse of Infix:

$(h+g)*(f+(e+d)+c+(b+a))$

Scan	stack	out put
(	(	-
h	(	h
+	(+	h

g	(+	hg
)	-	hg +
*	*	hg +
(	* (	hg +
f	* (	hg + f
+	* (+	hg + f
(	+ (+ (	hg + f
e	* (+ (	hg + f e
+	* (+ (+	hg + f e
d	* (+ (+	hg + f e d
)	* (+	hg + f e d +
*	* (+ *	hg + f e d +
(	* (+ * (	hg + f e d +
+	* (+ * (+	hg + f e d + c
(	* (+ * (+ (	hg + f e d + c + +
b	* (+ * (+ (	hg + f e d + c + + b
+	* (+ * (+ (+	hg + f e d + c + + b
a	* (+ * (+ (+	hg + f e d + c + + b a
)	* (+ *	hg + f e d + c + b a +
)	*	hg + f e d + c + b a + +
End	-	hg + f e d + c + b a + + *

final postfix:

hg + fed + (* + ba + + * *

Final prefix :

reverse (post fix)

=) * + + ab * c + def + gh

b) Infix to post fix

Scan	stack	out put
(	(	-
(	((	-
(	(((	-
a	(((	a
+	(((+	a
b	(((+	ab
)	((	ab +
+	((+	ab +
c	((+	ab + c
*	((+ *	ab + c
(	((+ * (	ab + c
d	((+ * (	ab + c d
+	((+ * (+	ab + c d

e	((+ * (→	ab + cde
)	((+ *	ab + cde +
+	((+ *	ab + cde + * +
f	((+	ab + cde + * + f
)	(	ab + cde + * + f +
*	(*	ab + cde + * + f +
(	(* (	ab + cde + * + f +
g	(* (	ab + cde + * + f + g
+	(* (+	ab + cde + * + f + g
h	(* (+	ab + cde + * + f + g h
)	(*	ab + cde + * + f + g h +
)	(	ab + cde + * + f + g h + *

final postfix:

ab + cde + * + f + g h + *

⑤ i) Fairness:

→ Simple Queue: Enforces strict temporal fairness (First in, first out).

→ Priority Queue: Trades strict arrival fairness for importance. Introduces starvation risk for non-VIP customers if VIP arrivals are continuous.

ii) Waiting time:

→ VIP: decreases dramatically.

→ normal: increases proportional to the rate of VIP arrivals.

iii) Computational complexity

a) Simple Queue: $O(1)$ Enqueue & dequeue

b) $O(\log n)$ enqueue & dequeue

→ $O(n)$ space complexity for both